

OSI modellen og TCP protokollen

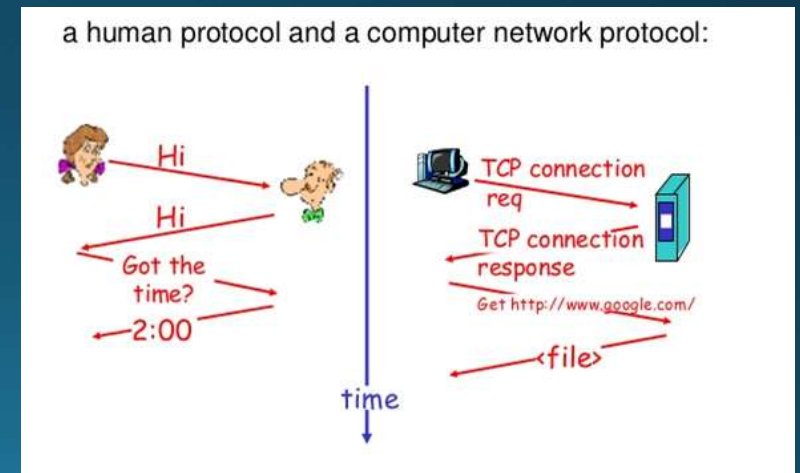
Netværk og distribueret design

Dagsorden

- Protokol
- OSI modellen
 - MAC adresser
 - IP-adresser
 - Routere
 - NAT
 - Porte/socket
- Øvelse og pause
- TCP/IP
- TCP og serialiseret data
- Tcp øvelse.

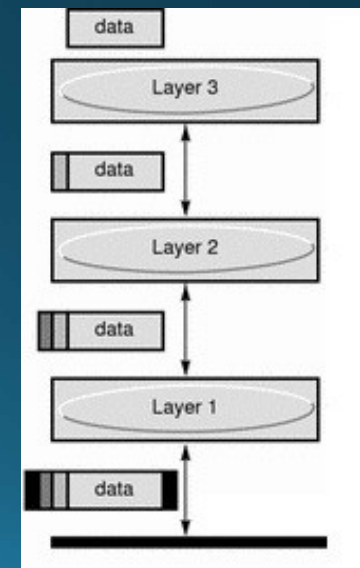
Protokol

- En protokol er et sæt regler, som beskriver hvordan et system kan kommunikere.
 - På et netværk beskriver det hvordan data overføres fra et system til et andet.
 - Inkluderer http, https, dns, tcp, udp, ftp.
 - Kan også beskrive hvordan data skal læses.



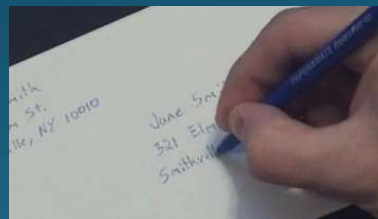
Protokol Stacks

- Flere protokoller som arbejder sammen, kaldes en protokol stack
 - Hvert lag af stakken udfører en bestemt funktion
 - Ved at lægge dem i lag, bliver systemet mere robust og pålidelig
 - Hver lag kommunikerer med det lag, som ligger over og under laget



Protokol Stacks

- Eksempel
 - En post service, består af følgende:
 - Afsenderen
 - Pakke ind
 - Adressere pakken
 - Aflevere pakken til postbuddet



Protokol Stacks

- På hovedkontoret
 - Pakker modtages
 - Pakker sorteres
 - Pakkerne flyttes diverse biler, som køres til andre afdelinger
- På det eksterne kontor
 - Pakkerne modtages
 - Pakker sorteres
 - Postbud leverer pakker

Protokol Stacks

- På modtager siden
 - Tage i mod pakken fra leverandøren
 - Pakke ud



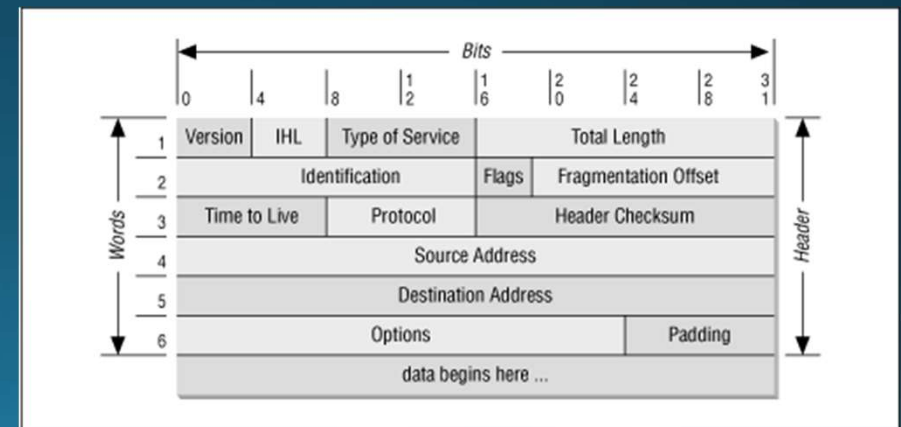
Protokol Stacks

- Hvorfor giver det mening at dele opgaverne op?
 - Hvert lag/opgave kan ændres uden at det har indvirkning på de andre opgaver
 - F.eks. Hvis vi vil bruge en scooter i stedet for en bil til at transportere pakkerne, vil det ikke have indflydelse på resten af opgaverne
 - Den interne post er ved ikke at transportmetoden har ændret sig
 - (desuden er den også ligeglad med det)



Pakke

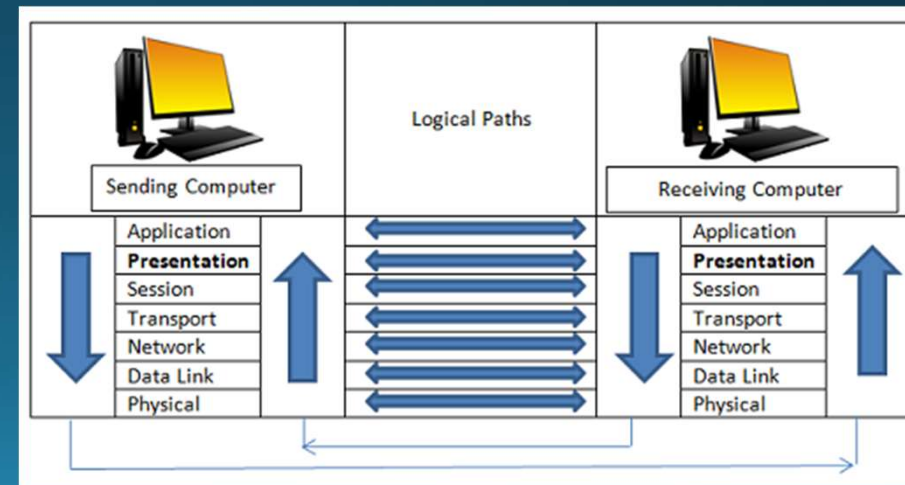
- Hvad er en 'pakke' i netværksforstand?
- En dataenhed, som bruges til at sende data over et netværk
 - Indeholder informationer om
 - Afsender, modtager, kontrolinformation mm.



IP pakke

OSI Modellen

- The Open System Interconnection model (OSI)
 - Definerer et netværks framework, som implementerer protokoller i 7 lag
 - Kommunikation bevæger sig fra et lag til det næste
 - Starter ved applikationslaget i en station
 - Bevæger sig til det nederste lag
 - Bevæger over en channel til den næste station
 - Bevæger sig fra bunden til toppen i den næste station
 - Med andre ord:
 - Sender: går vi ned
 - Modtager: går vi op



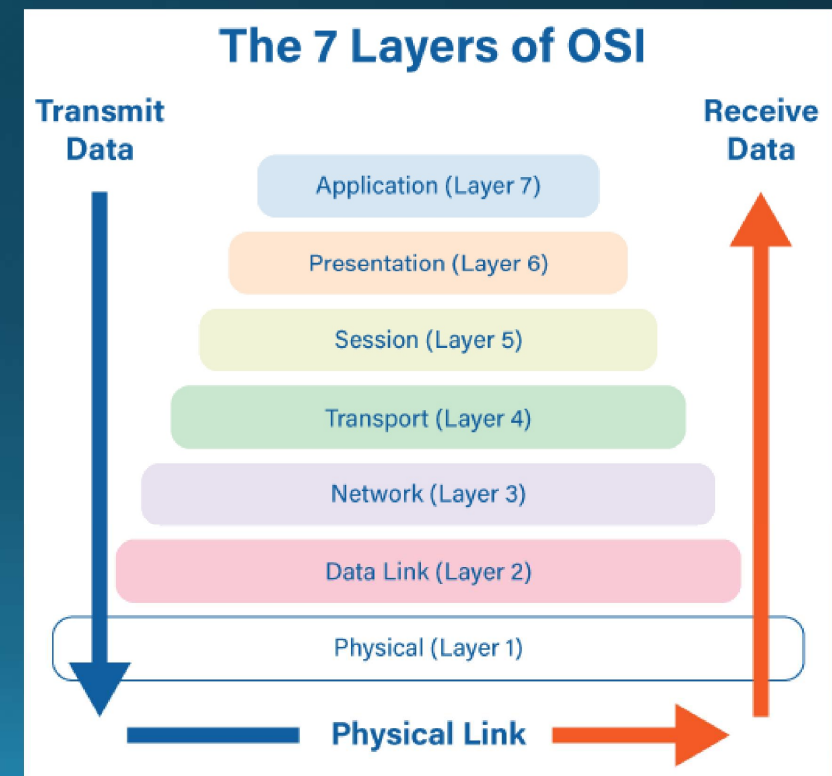
Tip til at huske det: Alle Præsidenter Skal Tæskes Når De Fødes

OSI Modellen

- OSI modellen er ikke håndgribelig
 - Har ingen funktionalitet
 - Det er et konceptuelt framework, som bruges til at standardisere kommunikationen på et computersystem uden at tage hensyn til den underliggende teknologi.
 - Ingen moderne protokoller implementerer den 1 til 1
 - Flere lag kan være kombineret eller helt unladt

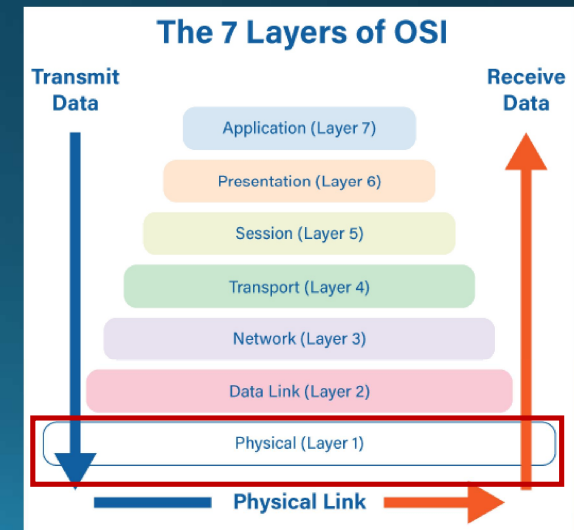
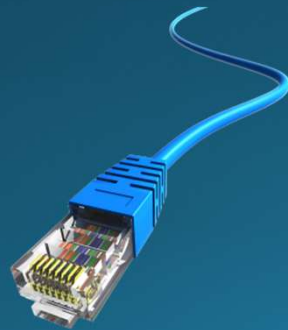
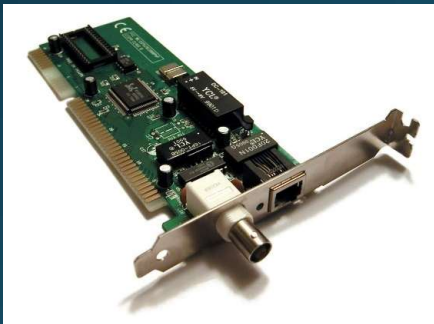
OSI Modellen

- OSI Modellen er delt op i en "vertical stack", som består af 7 lag
 - Data har forskellige navne på det forskellige stadier.
- Nede fra og op:
 - L1: bits
 - L2: frames
 - L3: packets
 - L4: segments



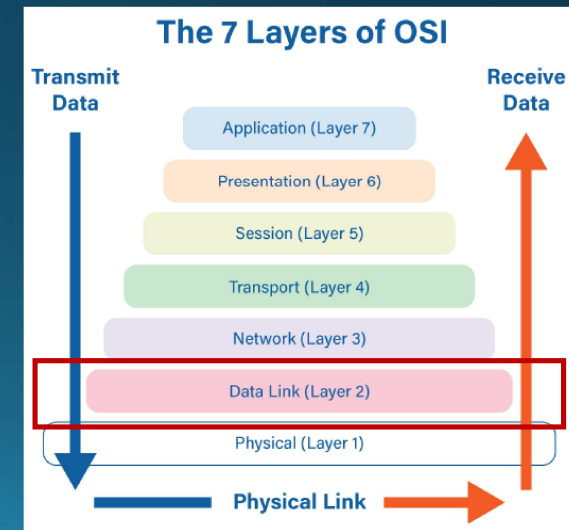
Physical (layer 1)

- Det laveste lag i OSI modellen
- Står for at sende og modtage ustrukturerede bit streams via et fysisk medie.
 - Konvertering af fysisk (strøm) til digitalt (bits)
- Den fysiske forbindelse mellem enhederne på netværket.
 - Dette inkluderer:
 - Kablede forbindelser –ethernet/fiber
 - Herindunder hubs/repeaters
 - Trådløse forbindelser såsom bluetooth eller wifi.



Data Link (Layer 2)

- Snakker med det fysiske lag og konvertere bits til frames
 - Og omvendt, den anden vej
- Fejlkontrol
 - Kan tjekke om frames er korrupte (ud fra bits i frame trailer)
- Benytter MAC adresser
 - Som afsender/modtager i frame header
 - Local Scope
 - Bruges kun i et lokalt netværk og ikke på tværs af forskellige netværk



MAC adresser

- Media Access Control adresse
- Unikt ID, der tildeles hver netværks enhed (f.eks netkort, bluetooth modtager/sender)
- Tildes af fabrikanten af den fysiske enhed
- 48 bits(6 bytes) i størrelse
- Skrives normalt i hexadecimal
- De første 3 bytes er kaldes OUI (Organizationally Unique Identifier)
 - Id'et fra fabrikaten (intel, asus etc.)
- De sidste 3 er ID tildelt til dette specifikke Medie.



MAC adresser

- Vi kan se vores egen mac adresse(r) i windows via følgende kommando i CLI

```
ipconfig /all
```

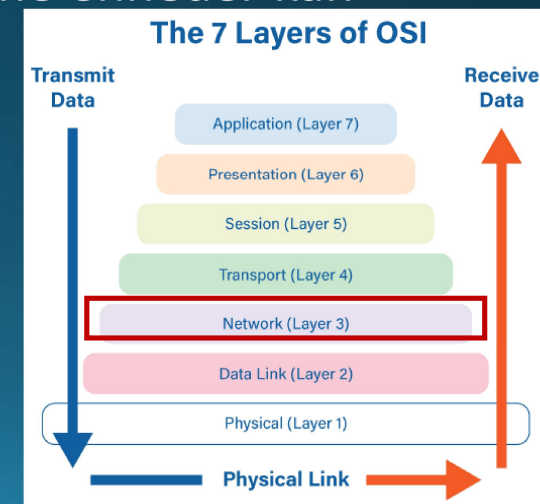
- Her ses mac adressen for et trådløst netkort, der er forbundet til eadania
- "Bo-A4-60" er intels OUI
 - Kan slåes op her
 - <https://hwaddress.com/oui-iab/Bo-A4-60/>
- "05-35-2A"
 - er id'et til præcis mit trådløse netkort

Wireless LAN adapter Wi-Fi:

```
Connection-specific DNS Suffix . : eadania.net.local
Description . . . . . : Intel(R) Wi-Fi 6 AX200 160MHz
Physical Address. . . . . : B0-A4-60-05-35-2A
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . : fe80::50ef:8fbc:b74f:ab8%18(Preferred)
IPv4 Address. . . . . : 10.131.65.65(Preferred)
Subnet Mask . . . . . : 255.255.240.0
Lease Obtained. . . . . : 25. maj 2023 09:07:31
Lease Expires . . . . . : 26. maj 2023 02:39:22
Default Gateway . . . . . : 10.131.64.1
DHCP Server . . . . . : 10.2.0.252
DHCPv6 IAID . . . . . : 129016928
DHCPv6 Client DUID. . . . . : 00-01-00-01-29-F4-18-DB-8C-8C-AA-02-AC-0D
DNS Servers . . . . . : 10.2.0.9
                        10.2.0.10
Primary WINS Server . . . . . : 10.2.1.12
Secondary WINS Server . . . . . : 10.2.1.11
NetBIOS over Tcpip. . . . . : Enabled
```

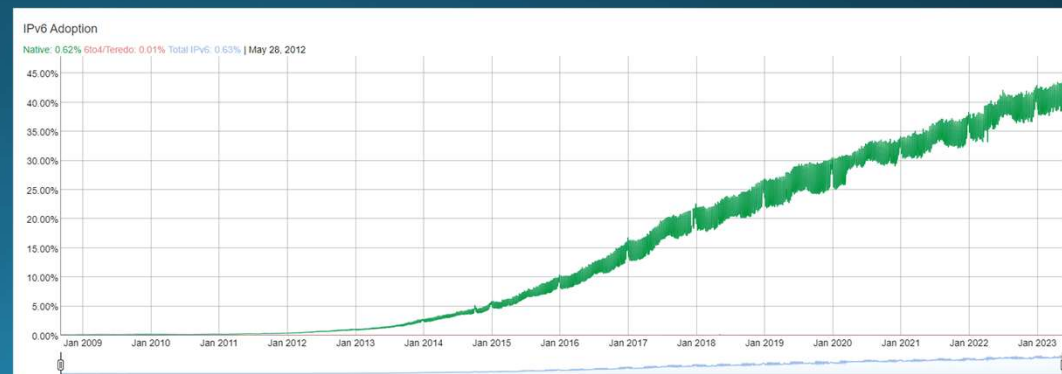

OSI Modellen Network (Layer 3)

- Ansvarlig for at håndtere routing og bestemme den optimale rute, som dataen skal tage gennem netværket
- Dette gøres i gennem IP-adresser, som bliver brugt til at finde vej gennem forskellige routere, og dermed bestemme den endelige rute.
 - AKA finde vej på netværket, f.eks internettet! (modsat MAC adresser)
- Bruger Network Address Translation (NAT) til at Flere forbundne enheder kan dele en ekstern Ip adresse
- Oversætter frames til packets
 - ...og omvendt, den anden vej 😊



IP Adresser

- IP-adresse står for Internet Protocol-adresse.
- Det er en numerisk identifikator til enheder tilsluttet et netværk.
- Der findes to versioner: IPv4 og IPv6.
- IPv4 består af 32 bits, normalt repræsenteret som fire decimaltal adskilt af punktummer.
 - 255.255.255.255
- IPv6 består af 128 bits, normalt repræsenteret som otte grupper af fire hexadecimaltal adskilt af kolon.
 - 2001:db8:a0b:12fo::0:0:1



IP Adresser

- IP-adresser bruges til at identificere afsender og modtager i netværkskommunikation.
- Offentlige IP-adresser er unikke og kan nås fra internettet.
 - Se evt. www.myip.com for at se jeres egen!
 - Muligvis den samme, hvis i er på dania nettet 😊
- Private IP-adresser bruges i private netværk og er ikke direkte tilgængelige fra internettet.
- IP-adresser anvendes af routere til "route" pakker i den rigtige retning og forhåbentligt til modtageren

Find egen IP - lokalt

- Find vores egen lokal tildelte ip på det netværk vi er på
 - cmd → ipconfig
- Routeren vi er forbundet til, tildeler os en lokal IP
 - "Default Gateway" er ip'en til vores Router
- Ikke sikkert vi får den samme adresse tildelt hver gang

Ethernet adapter Ethernet:

```
Connection-specific DNS Suffix . : localdomain
Link-local IPv6 Address . . . . . : fe80::6dcc:e4e8:516:69fd%18
IPv4 Address. . . . . : 192.168.1.2
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
```

Find egen IP - lokalt

- cmd → ping
 - F.eks. ping google.com eller ping 192.168.1.1
- Se om i kan pinge hinandens adresser
- Værd at vide omkring ip:
 - Localhost = 127.0.0.1 = loopback adresse = egen maskine!

```
C:\Users\Jan Philip Tsanas>ping google.com

Pinging google.com [142.250.184.206] with 32 bytes of data:
Reply from 142.250.184.206: bytes=32 time=13ms TTL=54
Reply from 142.250.184.206: bytes=32 time=13ms TTL=54
Reply from 142.250.184.206: bytes=32 time=12ms TTL=54
Reply from 142.250.184.206: bytes=32 time=57ms TTL=54

Ping statistics for 142.250.184.206:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 12ms, Maximum = 57ms, Average = 23ms
```

TTL = Time To Live

Routerne

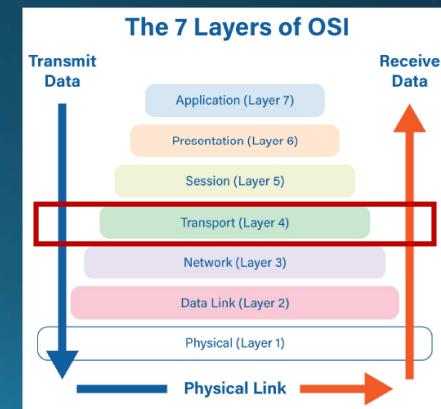
- Modtager en udadgående IP-adresse fra internettjenesteudbyderen (ISP) og bruger den som deres offentlige IP-adresse.
- Routerne tildeler lokale IP-adresser til de enheder, der er tilsluttet det lokale netværk.
- Routerne opretholder routingtabeller og bruger routingalgoritmer til at finde den optimale rute for dataene.
 - Dvs. holder styr på hvem der er connected/hvor de er.
- Routerne fungerer som gateways og dirigerer data mellem forskellige netværk.
 - En router kender til en anden, som kender til en anden, som kender til en anden...

NAT

- NAT (Network Address Translation) udføres af routeren for at tillade flere enheder at dele en offentlig udadgående IP-adresse.
- Havde vores routere ikke haft Nat ville IPv4(32bit) ikke være tilstrækkeligt
 - ~4.3 milliader Adresser er ikke helt nok til at holde på alle enheder på internettet i dag.
- Vi er stille og roligt(siden ~2010) begyndt at skifte til IPv6, hvor vi har 128bit forskellige adresser
 - $2^{128} = 340,282,366,920,938,463,374,607,431,768,211,456$
 - 340 undecillioner adresser!
- I teorien ville fuld implementering af IPv6 eliminere behovet for NAT, da der ville være nok adresser til at tildele enheder direkte uden behov for IP-adresseoversættelse.

OSI Modellen Transport (Layer 4)

- Sørger for at beskeder leveres uden fejl, uden pakke tab og uden duplikationer.
 - Højere lag skal derfor ikke bekymre sig om data overførsel.
- Komplexiteten af transportlaget er afhængig af netværkslaget
 - Hvis netværkslaget er pålideligt, skal transportlaget ikke håndtere meget.
 - Hvis netværkslaget der imod er upålideligt, skal transportprotokollen gøre op for det.
- Primært 2 protokoller
 - Tcp
 - "Langsom" og reliable
 - Udp
 - Hurtig og unreliable
- Bruger Porte til at identificere specifikke applikationer/processer
Processor eller programmer på computeren



Porte

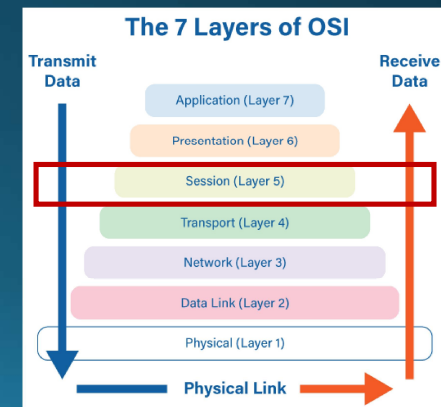
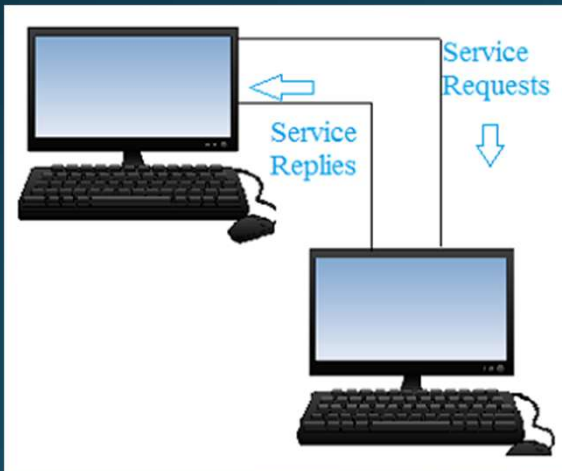
- Porte er numeriske identifikatorer, der bruges til at specificere, hvilken applikation eller tjeneste der skal modtage data på en netværksforbindelse.
- Porte er en del af transportlaget i OSI-modellen.
- Der er 65.535 (2byte) porte til rådighed, hvoraf nogle er reserveret til specifikke protokoller eller anvendelser.
 - 80 (HTTP), 443 (HTTPS), 25 (SMTP) og 53 (DNS).
- Portnumre under 1024 er kendt som velkendte porte og kræver normalt administratorrettigheder til at blive anvendt.
 - Brug derfor IKKE noget under 1024 til jeres egne applikationer

Porte

- En enkelt IP-adresse kan have flere porte åbne samtidigt for forskellige applikationer eller tjenester.
 - Dvs at man kan "lytte" på en port, så man er klar til at modtage på denne port.
- Flere applikationer kan IKKE bruge samme port på samme maskine.
- Når en enhed sender data til en anden enhed over netværket, angiver den både en IP-adresse og en portnummer kombination for at identificere den specifikke destination.
 - Til sammen giver denne kombination et Socket

OSI Modellen Session (Layer 5)

- Dette lag etablerer, styrer og afslutter sessioner (forbindelser) mellem enheder.
 - Tillader at to applikationer på forskellige maskiner at oprette en forbindelse, kaldet en session.
 - Tillader sessionerne at kommunikerer over netværket.
 - Den reele opgave for session laget afhænger af den brugte protokol UDP eller TCP

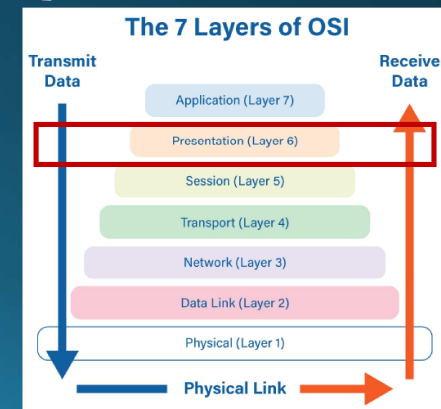


OSI Modellen Presentation (Layer 6)

- Formaterer data, så det kan præsenteres for applikationslaget.
- Man kan se på det som en oversætter.
- Sender: Oversætter fra applikationsformat til et generelt format.
- Modtager: Oversætter fra et generelt format til et applikationsformat.
 - Konvertering; Serialisering, encoding, komprimering, kryptering osv.

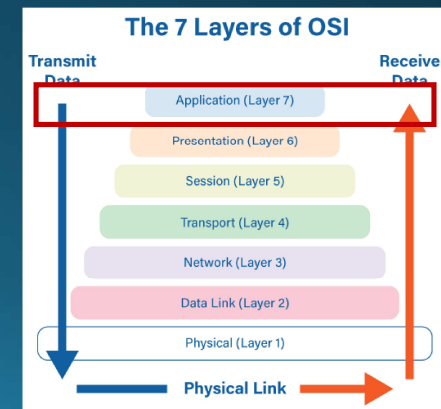
Serdialisering og Deserialisering

Gøre det klar til næste lag



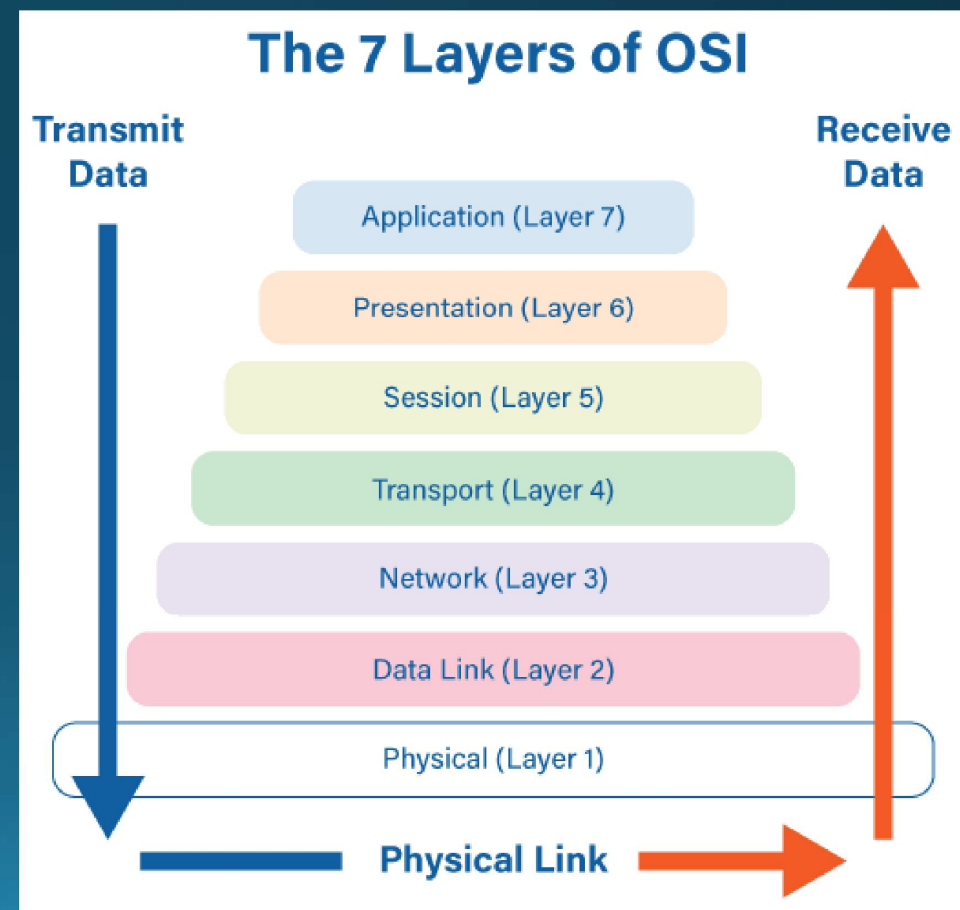
OSI Modellen Application (Layer 7)

- Applikationen, som brugeren benytter sig af.
- Dvs en browser, et program, et spil etc.
- Håndterer funktioner som forbindelse til netværksfiler osv.
 - En slags interface til netværk
 - Browsers bruger f.eks DNS og HTTP
 - Email bruger POP og SMTP



OSI Modellen opsummering

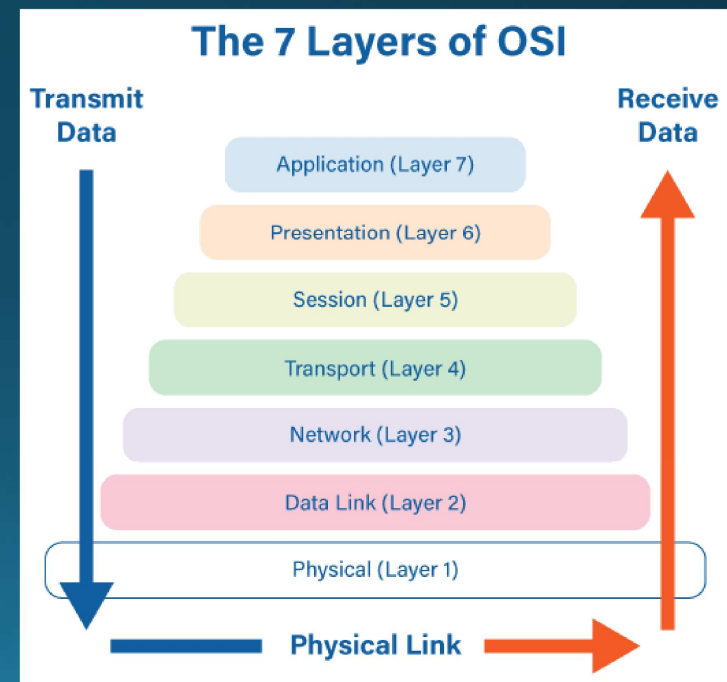
- **Applikationslag:** Brugerapplikationer og tjenester, der giver specifik funktionalitet til brugeren, som f.eks. e-mail, webbrowsing og filoverførsel.
- **Præsentationslag:** Håndterer dataformatering, kryptering, komprimering og konvertering mellem forskellige datarepræsentationer.
- **Sessionlag:** Etablerer, styrer og afslutter sessioner mellem applikationer, herunder autentifikation og sikkerhed.
- **Transportlag:** Sørger for pålidelig og fejlfri overførsel af data mellem applikationer ved hjælp af TCP eller UDP.
- **Netværkslag:** Styrer routing af datapakker gennem netværket ved hjælp af IP-adressering og routingprotokoller.
- **Datalink lag:** Etablerer forbindelse mellem direkte forbundne enheder og håndterer fejldetektering og rettelser.
- **Fysisk lag:** Overfører rå bitstrøm mellem enheder gennem fysiske medier som kabler og signaler.



Øvelse og pause (~20 min)

- Nedenfor er nogle netværks termer og protokoller

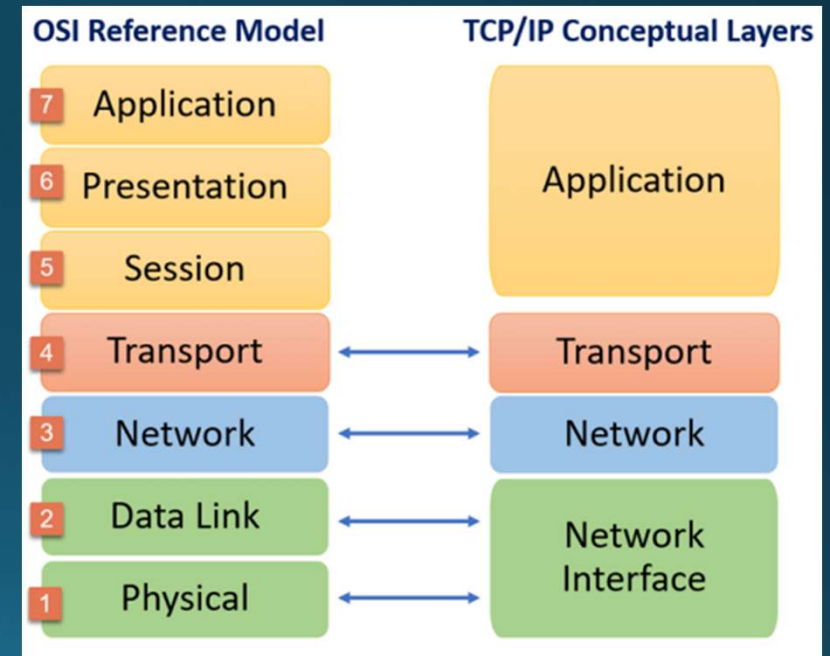
- HTTP = Application
- IP-adresse = Netværk
- MAC-adresse = Data
- Fiber kabel = Fysisk
- TCP = Transport
- HTTPS = Application
- SMTP = Application
- Portnummer = Transport
- UDP = Transport
- WIFI = Fysisk
- Ethernet kabel = Fysisk



TCP/IP

- OSI modellen er den teoretiske model
 - Ikke en konkret implementation
- TCP/IP er det vi faktisk bruger
 - Simplere model (kun 4 lag)
 - Samme princip

Nå vi snakker lag, refererer vi til OSI modellen



TCP/IP

- TCP: Transmission control protocol
- IP: Internet protocol
- Tilsammen udgør de "backbone" af næsten alt hvad du foretager dig online.
 - Web browsing, email etc. (er bygget oven på TCP/IP)



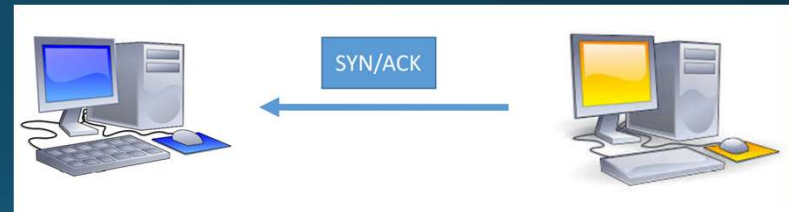
TCP/IP

- TCP er en pålidelig "connection based" protokol.
- Dvs. Der oprettes en forbindelse mellem to computere.
- Herefter sendes der data mellem computerne – stream baseret.
 - At sende fungerer som at skrive til en fil
 - At modtage fungerer som at læse en fil



TCP/IP - 3-Way Handshake

- 3-Way Handshake
- 1. En client sender en SYN(synchronization) datapakke til en server over et IP netværk.
 - Formålet med pakken er at bede om en forbindelse
- 2. Serveren svarer med en SYN/ACK pakke
- 3. Klienten modtager SYN/ACK pakken og svarer med ACK pakken.
 - Når dette er forløbet godt, begynder klienten og serveren at kommunikere gennem den valgte socket (IP/Port), som nu er låst.
- Kun når forbindelse oprettes, ikke hver gang!



TCP/IP

- Forbindelsen er
 - Pålidelig og sorteret.
 - Det data du sender er garanteret at komme frem på den anden side i den samme rækkefølge, som det blev sendt.
- Stream af data
 - TCP opdeler dit data i pakker og sender det over netværket.
- Husk det er ligesom at skrive til en fil via en stream!
 - Vi skriver bare til en anden maskine, ikke en fil

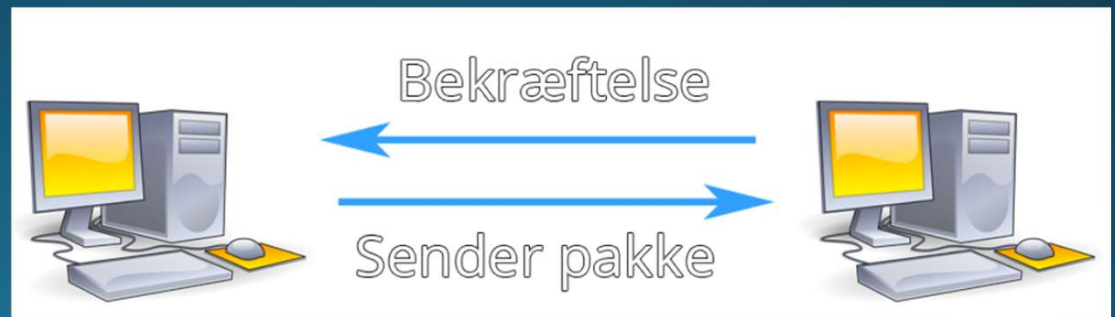
TCP

- TCP deler en stream af data op i pakker
 - Disse pakker sendes over en upålidelig IP
 - Pakkerne samles igen på modtager siden
 - Der bliver sendt en ack pakke efter hver modtaget pakke, så ved at den kom godt frem



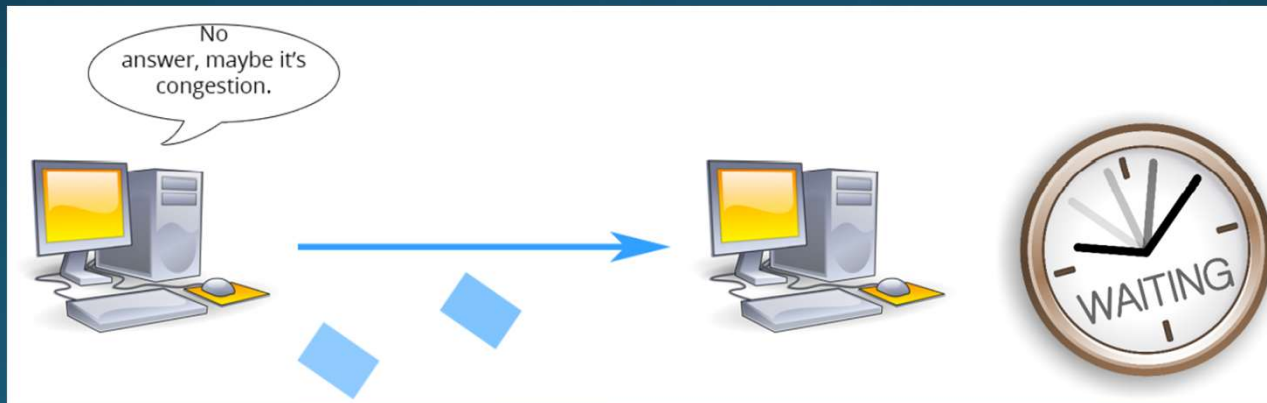
TCP

- Men hvad sker der når vi:
- Mister en pakke
 - Pakken sendes igen hvis der ikke kommer en ack(bekræftelse) tilbage
- Hvis pakker ankommer i forkert rækkefølge
 - Pakkerne samles i den korrekte rækkefølge
- Hvis pakker er duplikeret
 - Pakkerne smides væk



TCP

- Dvs. at vi skal vente på at en pakke sendes igen hver gang en pakke går tabt.
 - Nyt data bliver smidt i kø, og kan ikke bruges før den tabte pakke ankommer.
 - Hvad hvis den næste pakke også går tabt?
- Derfor kan man komme til at vente længe på pakker. (baseret på din ping)
 - Desuden kan det være at TCP beslutter sig for at der er tale om congestion, og venter med at sende pakker.



TCP

- TCP bør derfor ikke bruges i sammenhæng med multiplayer spil hvor realtime data er nødvendigt.
 - Hurtigt betyder her hvor ofte der skal sendes pakker, ikke hvor hurtigt spillet går.
 - F.eks FPS = hurtig
 - Diablo = langsom
 - Vi er nemlig ligeglade med hvad der skete for et sekund siden.
 - Vi er kun interesseret i det nyeste data.
 - TCP var ikke designet til at håndtere dette
 - Tcp kan fungerer rigtig godt til "langsomme" spil, eller en chat applikation.

TCP Serveren

- Simpleste form for server
- Tager imod hvilken som helst client på port 12000
- Laver en tråd pr connectede client.
- Echoer blot strings med suffixet "From Server"
- Alle indstillinger er blot på standard tcp.

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Opretter en TcpListener-objekt, der lytter på alle tilgængelige IP-adresser på port 12000.
- Når den er startet er denne server nu klar til at modtage tcp pakker sendt til adressen for maskinen fra en vilkårlig Ip adresse på port 12000.
- Starter TCP-serveren og begynder at lytte efter indkommende forbindelser.
- Mere om TcpListener her:
 - <https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcplisten-er?view=net-7.0>

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Starter en uendelig løkke for at acceptere klientforbindelser.
- Venter og accepterer en indgående TCP-forbindelse fra en klient.
 - OBS: Blokerende!
 - Venter derfor på at klienter connecter
- Opretter og starter en ny tråd til at håndtere klient forbindelse(r)

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient(); Venter her, hvis der ikke er nogen for forbinder
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Kode der håndterer hver enkelt klient (en for hver tråd)
- Starter med at lave et unikt ID til klienten
 - Dette er blot så der kan kendes forskel på forskellige klienter
- Udskriver dernæst ID'et på den connectede klient

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Opretter en StreamWriter til at skrive data til klientens stream.
- Opretter en StreamReader til at læse data fra klientens stream.
- Selve streamen kan forstås som et delt objekt mellem server og klient

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                // Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Starter en løkke for at modtage og sende beskeder, så længe klienten er tilsluttet.
- Læser en linje fra klienten, der er sendt som en besked.
 - Obs: Det er en stream, så hvornår er beskeden slut?
 - ReadLine() – Så den venter på en "\n"
- Kontrollerer, om beskeden ikke er null.
- Skriver beskeden sammen med suffixet "From server" tilbage til klientens stream.
- Tømmer bufferen for at sikre, at dataene sendes til klienten.
 - Dvs. den lokale writer nu er tømt

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Serveren

- Fanger eventuelle exceptions, der kan opstå under behandlingen af klienten.
- Udskriver en fejlmeddelelse på konsollen, der angiver klientens ID og fejlmeddelelsen.
 - Rig mulighed for at notere hvilke type exception der kan ske og håndtere disse individuelt
- Sørger i Finally blokken at frigive ressourcerne forbundet med klienten.

```
using System.Net.Sockets;
using System.Net;

TcpListener server = new TcpListener(IPAddress.Any, 12000);
server.Start();
Console.WriteLine("Server started... listening on port 12000");

while (true)
{
    TcpClient client = server.AcceptTcpClient();
    Thread clientThread = new Thread(() => HandleClient(client));
    clientThread.Start();
}

void HandleClient(TcpClient client)
{
    Guid clientId = Guid.NewGuid();
    Console.WriteLine($"Client {clientId} connected.");
    // Send client name to the client
    StreamWriter writer = new StreamWriter(client.GetStream());
    // Receive and send messages
    StreamReader reader = new StreamReader(client.GetStream());
    try
    {
        while (client.Connected)
        {
            string message = reader.ReadLine();
            if (message != null)
            {
                Console.WriteLine($"Received message from {clientId}: {message}");
                //Echoing message
                writer.WriteLine(message + " From server");
                writer.Flush();
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception occurred for client {clientId}: {ex.Message}");
    }
    finally
    {
        Console.WriteLine($"Client disconnected: {clientId}");
        client.Dispose();
    }
}
```

TCP Klienten

- Endnu simplere
- Connector til en server på localhost port 12000 Skal være den samme som serveren
- Laver forbindelse, og laver en receive tråd, mens main threaden kan læse fra consollen.
- Sender blot input til serveren og forventer et svar fra serveren
 - Note: Ingen fejl sikring på denne.
- Mere om TcpClient her:
<https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.tcpclient?view=net-7.0>

```
using System.Net.Sockets;

TcpClient client = new TcpClient();
client.Connect("localhost", 12000);
Console.WriteLine("Connected to server...");

// Start a thread to receive messages
Thread receiveThread = new Thread(() => ReceiveMessages(client));
receiveThread.Start();

// Send messages
StreamWriter writer = new StreamWriter(client.GetStream());
while (true)
{
    string message = Console.ReadLine();
    writer.WriteLine(message);
    writer.Flush();
}

void ReceiveMessages(TcpClient client)
{
    StreamReader reader = new StreamReader(client.GetStream());
    while (client.Connected)
    {
        string message = reader.ReadLine();
        if (message != null)
        {
            Console.WriteLine(message);
        }
    }
}
```


TCP Klienten

- Opretter en TCPClient-objekt til at oprette forbindelse til en server.
- Opretter forbindelse til serveren på localhost (den lokale maskine) på port 12000.
 - Skulle her connecte over internet / netværk, skulle vi bruge en anden adresse end "localhost"
 - Localhost er kendt som "loopback Adressen"
 - Er det samme som 127.0.0.1

```
using System.Net.Sockets;

TcpClient client = new TcpClient();
client.Connect("localhost", 12000);
Console.WriteLine("Connected to server...");

// Start a thread to receive messages
Thread receiveThread = new Thread(() => ReceiveMessages(client));
receiveThread.Start();

// Send messages
StreamWriter writer = new StreamWriter(client.GetStream());
while (true)
{
    string message = Console.ReadLine();
    writer.WriteLine(message);
    writer.Flush();
}

void ReceiveMessages(TcpClient client)
{
    StreamReader reader = new StreamReader(client.GetStream());
    while (client.Connected)
    {
        string message = reader.ReadLine();
        if (message != null)
        {
            Console.WriteLine(message);
        }
    }
}
```

TCP Clienten

- Opretter og starter en ny tråd til at modtage beskeder fra serveren.
- Serveren hedder i dette tilfælde "client" da den er af typen tcpclient.

```
using System.Net.Sockets;

TcpClient client = new TcpClient();
client.Connect("localhost", 12000);
Console.WriteLine("Connected to server...");

// Start a thread to receive messages
Thread receiveThread = new Thread(() => ReceiveMessages(client));
receiveThread.Start();

// Send messages
StreamWriter writer = new StreamWriter(client.GetStream());
while (true)
{
    string message = Console.ReadLine();
    writer.WriteLine(message);
    writer.Flush();
}

void ReceiveMessages(TcpClient client)
{
    StreamReader reader = new StreamReader(client.GetStream());
    while (client.Connected)
    {
        string message = reader.ReadLine();
        if (message != null)
        {
            Console.WriteLine(message);
        }
    }
}
```

TCP Klienten

- Opretter en StreamWriter til at skrive data til serverens stream.
- Starter en uendelig løkke for at indtaste og sende beskeder til serveren
- Læser en linje fra konsollen, der skal sendes som en besked til serveren.
- Skriver beskeden til serverens stream.
- Tømmer bufferen for at sikre, at dataene sendes til serveren.

```
using System.Net.Sockets;

TcpClient client = new TcpClient();
client.Connect("localhost", 12000);
Console.WriteLine("Connected to server...");

// Start a thread to receive messages
Thread receiveThread = new Thread(() => ReceiveMessages(client));
receiveThread.Start();

// Send messages
StreamWriter writer = new StreamWriter(client.GetStream());
while (true)
{
    string message = Console.ReadLine();
    writer.WriteLine(message);
    writer.Flush();
}

void ReceiveMessages(TcpClient client)
{
    StreamReader reader = new StreamReader(client.GetStream());
    while (client.Connected)
    {
        string message = reader.ReadLine();
        if (message != null)
        {
            Console.WriteLine(message);
        }
    }
}
```

TCP Klienten

- Opretter en StreamReader til at læse data fra serverens strøm.
- Starter en løkke for at modtage beskeder fra serveren, så længe klienten er tilsluttet.
- Læser en linje fra serveren, der er sendt som en besked.
- Kontrollerer, om beskeden ikke er null og udskriver til konsol hvis dette ikke er tilfældet.

```
using System.Net.Sockets;

TcpClient client = new TcpClient();
client.Connect("localhost", 12000);
Console.WriteLine("Connected to server...");

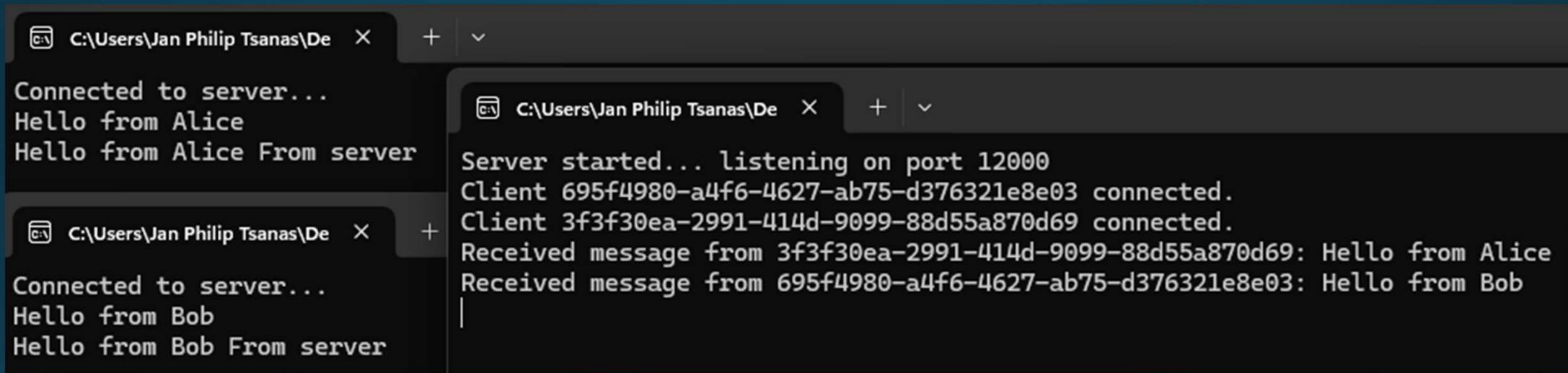
// Start a thread to receive messages
Thread receiveThread = new Thread(() => ReceiveMessages(client));
receiveThread.Start();

// Send messages
StreamWriter writer = new StreamWriter(client.GetStream());
while (true)
{
    string message = Console.ReadLine();
    writer.WriteLine(message);
    writer.Flush();
}

void ReceiveMessages(TcpClient client)
{
    StreamReader reader = new StreamReader(client.GetStream());
    while (client.Connected)
    {
        string message = reader.ReadLine();
        if (message != null)
        {
            Console.WriteLine(message);
        }
    }
}
```

TCP Serveren - eksempel

Client



```
C:\Users\Jan Philip Tsanas\De  X + v
Connected to server...
Hello from Alice
Hello from Alice From server

C:\Users\Jan Philip Tsanas\De  X +
Connected to server...
Hello from Bob
Hello from Bob From server

C:\Users\Jan Philip Tsanas\De  X + v
Server started... listening on port 12000
Client 695f4980-a4f6-4627-ab75-d376321e8e03 connected.
Client 3f3f30ea-2991-414d-9099-88d55a870d69 connected.
Received message from 3f3f30ea-2991-414d-9099-88d55a870d69: Hello from Alice
Received message from 695f4980-a4f6-4627-ab75-d376321e8e03: Hello from Bob
|
```

Client

Server

TCP med binær data?

- Når vi bruger StreamWriter bliver strings konverteret til et binært array med “\n” til sidst
- Når vi modtager læses hver byte individuelt indtil “\n” læses, og det vurderes at være en linje.
- Siden Tcp er stream baseret er vi nødt til at kunne adskille vores data, hvis vi ønsker Forskellige besked typer.
- Der er 2 forskellige måder vi kan håndtere adskillelse af data på:
 1. Seperator
 1. Den løsning der bliver brugt med streamWriteren
 2. Længdeindikator
 1. Skriver et tal før selve beskeden der beskriver hvor lang dataen er

TCP med binær data?

- Skriver vi om til at i stedet sende binær data direkte får vi ingen indbygget seperator
- Vi benytter her clientens network stream direkte.
- Vi kunne jo principielt serialisere og deserialisere her, men der er et problem...

```
Connected to server...  
hello server!
```

```
NetworkStream stream = client.GetStream();  
while (true)  
{  
    string message = Console.ReadLine();  
    byte[] serializedData = Encoding.UTF8.GetBytes(message);  
    // Send the serialized data  
    stream.Write(serializedData, 0, serializedData.Length);  
    stream.Flush();  
}
```

Send

```
Server started... listening on port 12000  
Client 27b1bee8-c23e-4ade-bda3-02f0b151b056 connected.  
Recived hello server!  
|
```

```
NetworkStream stream = client.GetStream();  
while (client.Connected)  
{  
    byte[] buffer = new byte[1024]; Det er et gæt på, hvor lang ens besked er. Dvs  
//returns -1 if end of stream. behøver ikke 1024  
    int bytesRead = stream.Read(buffer, 0, buffer.Length);  
  
    if (bytesRead > 0)  
    {  
        string deSerializedData = Encoding.UTF8.GetString(buffer);  
        Console.WriteLine("Recived " + deSerializedData);  
    }  
}
```

modtag

TCP med binær data?

- Sender vi beskeder hurtigt går det galt.
- Serveren har ingen chance for at vide hvad der er tiltænkt at være samme besked, da det er en stream!

"Smelter" beskeder sammen => skal kunen seperarer

```
Connected to server...  
hello server!
```

```
NetworkStream stream = client.GetStream();  
while (true)  
{  
    string message = Console.ReadLine();  
    for (int i = 0; i < 10; i++)  
    {  
        byte[] serializedData = Encoding.UTF8.GetBytes(message);  
        // Send the serialized data  
        stream.Write(serializedData, 0, serializedData.Length);  
        stream.Flush();  
    }  
}
```

Send

```
Server started... listening on port 12000  
Client 6c0386fd-581a-42c6-922c-9fdbb20d9153 connected.  
Recived hello server!  
Recived hello server!hello server!hello server!hello server!  
rver!
```

```
NetworkStream stream = client.GetStream();  
while (client.Connected)  
{  
    byte[] buffer = new byte[1024];  
    //returns -1 if end of stream.  
    int bytesRead = stream.Read(buffer, 0, buffer.Length);  
  
    if (bytesRead > 0)  
    {  
        string deSerializedData = Encoding.UTF8.GetString(buffer);  
        Console.WriteLine("Recived " + deSerializedData);  
    }  
}
```

modtag

TCP med binær data?

- Vi kan vælge egen separator.
- ...ligesom det StreamWriter og StreamReader gør

Modtag

```
NetworkStream stream = client.GetStream();
byte[] separator = Encoding.UTF8.GetBytes("\n");
while (true)
{
    string message = Console.ReadLine();
    for (int i = 0; i < 10; i++)
    {
        byte[] serializedData = Encoding.UTF8.GetBytes(message);
        // Send the serialized data
        stream.Write(serializedData, 0, serializedData.Length);
        stream.Write(separator, 0, separator.Length);
        stream.Flush();
    }
}
```

Send

```
NetworkStream stream = client.GetStream();
MemoryStream messageBuffer = new MemoryStream(); // Buffer to hold partial messages
byte[] buffer = new byte[1024];
byte[] separator = Encoding.UTF8.GetBytes("\n"); // Separator as byte array
int bytesRead;
while ((bytesRead = stream.Read(buffer, 0, buffer.Length)) > 0)
{
    for (int i = 0; i < bytesRead; i++)
    {
        byte currentByte = buffer[i];

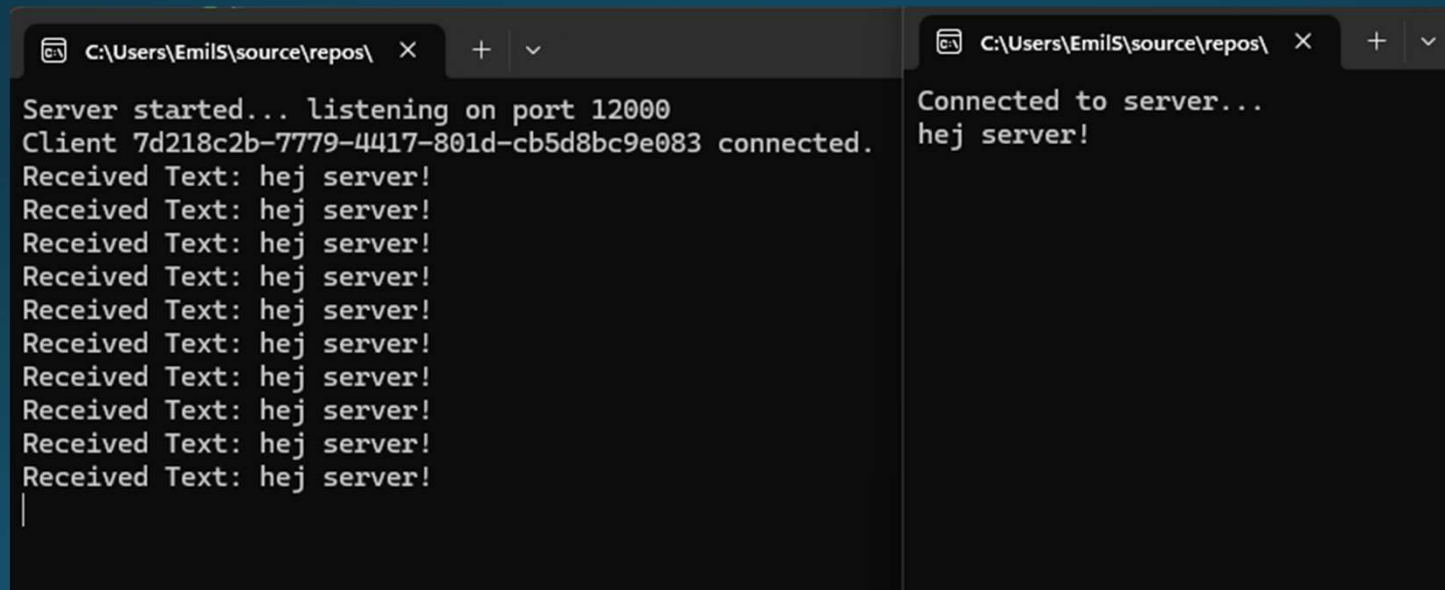
        if (currentByte == separator[0]) // Passer den med separator
        {
            // Process the received message
            byte[] rawData = messageBuffer.ToArray();
            string fullMessage = Encoding.UTF8.GetString(rawData);

            Console.WriteLine("Received Text: " + fullMessage);

            // Clear the buffer for the next message
            messageBuffer = new MemoryStream();
        }
        else
        {
            // Add the byte to the buffer
            messageBuffer.WriteByte(currentByte);
        }
    }
}
```

TCP med binær data?

- Beskeder er nu seperaret korrekt.



The image shows two side-by-side terminal windows. The left window is a command prompt titled 'C:\Users\EmilS\source\repos\' showing a server application running. It displays the message 'Server started... listening on port 12000', followed by 'Client 7d218c2b-7779-4417-801d-cb5d8bc9e083 connected.' and then ten lines of 'Received Text: hej server!'. The right window is also titled 'C:\Users\EmilS\source\repos\' and shows a client application running. It displays 'Connected to server...' followed by 'hej server!'.

```
C:\Users\EmilS\source\repos\ X + v
Server started... listening on port 12000
Client 7d218c2b-7779-4417-801d-cb5d8bc9e083 connected.
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
Received Text: hej server!
|

C:\Users\EmilS\source\repos\ X + v
Connected to server...
hej server!
```

TCP med binær data?

- Fordelen ved at sende og modtage rå bytes
 - Kan serialisere data, sende og deserialisere
 - ... men kræver vi selv laver funktionalitet til at separere beskeder
- StreamWriter og StreamReader
 - Giver os indbygget separation af beskeder
 - WriteLine / ReadLine
 - Sender og modtager strings, ikke rå bytes

TCP med binær data?

- Med længde indikator skriver vi blot en int før vores data
- Ved modtagelse, læser vi først denne int
 - Dvs. de 4 første bytes
- Dernæst kan vi oprette en buffer med korrekt længde og indlæse dataen

```
NetworkStream stream = client.GetStream();
while (true)
{
    string message = Console.ReadLine();
    byte[] serializedData = Encoding.UTF8.GetBytes(message);

    for (int i = 0; i < 10; i++)
    {
        // Send the length of the message as 4-byte integer
        stream.Write(BitConverter.GetBytes(serializedData.Length), 0, 4);
        stream.Write(serializedData, 0, serializedData.Length);
    }
    stream.Flush();
}
```

Send

```
NetworkStream stream = client.GetStream();
byte[] lengthBuffer = new byte[4];
while (client.Connected)
{
    int bytesRead = stream.Read(lengthBuffer, 0, 4);

    int messageLength = BitConverter.ToInt32(lengthBuffer, 0);
    byte[] buffer = new byte[messageLength];
    bytesRead = stream.Read(buffer, 0, buffer.Length);

    if (bytesRead > 0)
    {
        string receivedData = Encoding.UTF8.GetString(buffer);
        Console.WriteLine("Received: " + receivedData);
    }
}
```

modtag

TCP med binær data?

- Vi kan forkorte det ved at benytte BinaryReader og BinaryWriter
 - Converterer til og fra bytes for os

```
NetworkStream stream = client.GetStream();
BinaryWriter writer = new BinaryWriter(stream);

while (true)
{
    string message = Console.ReadLine();
    byte[] serializedData = Encoding.UTF8.GetBytes(message);
    for (int i = 0; i < 10; i++)
    {
        // Send the length of the message as 4-byte integer
        writer.Write(serializedData.Length);
        writer.Write(serializedData);

        stream.Flush();
    }
}
```

Send

```
NetworkStream stream = client.GetStream();
BinaryReader reader = new BinaryReader(stream);

while (client.Connected)
{
    int messageLength = reader.ReadInt32();
    byte[] buffer = reader.ReadBytes(messageLength);

    string receivedData = Encoding.UTF8.GetString(buffer);
    Console.WriteLine("Received: " + receivedData);
}
```

modtag

TCP med binær data?

- Vi kan bruge begge metoder hvis der ønskes
- Der kan også udvides
 - f.eks bruge både længde og en indikator til besked type, så vi ved hvordan der skal deserialiseres.

```
NetworkStream stream = client.GetStream();
BinaryWriter writer = new BinaryWriter(stream);

while (true)
{
    string message = Console.ReadLine();
    byte messageType = 0;
    byte[] serializedData = Encoding.UTF8.GetBytes(message);
    // Send the length of the message as 4-byte integer
    writer.Write(serializedData.Length);
    writer.Write(messageType);
    writer.Write(serializedData);

    stream.Flush();
}
```

Send

```
NetworkStream stream = client.GetStream();
BinaryReader reader = new BinaryReader(stream);

while (client.Connected)
{
    int messageLength = reader.ReadInt32();
    int messageType = reader.ReadByte();
    switch (messageType)
    {
        case 0:
            //handle message of type 0
            break;
        case 1:
            //handle message of type 2
            break;
        default:
            break;
    }
}
```

modtag

Seperator vs længdeindikator

- **Separator (Delimiter):**

- Bruger en speciel sekvens (delimiter) til at adskille beskeder.
- Almindeligt anvendt i tekstbaserede beskeder.
- Eksempler på delimiters: newline (\n), nulbyte (\0), semikolon (;).

- **Fordele:**

- Enkel implementering.
- Naturlig adskillelse af tekstbeskeder.
- Let at identificere beskeder med kendte separatore.

- **Ulemper:**

- Kan forårsage problemer, hvis delimiter indgår i beskedindholdet.

- **Længdeindikator:**

- Inkluderer en indikation af beskedens længde før selve beskeddataene.
- Giver præcis information om, hvor meget data der skal læses.

- **Fordele:**

- Fleksibel og robust.
- Kan håndtere beskeder af vilkårlig længde.
- Indikerer klart beskedens længde.

- **Ulemper:**

- Kan være mere komplekst at implementere end delimiter.
 - (f.eks. håndtering af, hvis mængde af data endnu ikke tilgængelig)

TCP Client/server Øvelse

- Udvid de 2 programmer (findes på moodle):
 - Network02-1 (template I skal arbejde ud fra)
 - Network02TCP – MessagePack (eksempel med besked-typer)
 - Network02TCP – SimpleSolution (simpel løsning)
 - Network02TCP – MultipleChatSolution (mere "produktionsklar" løsning)
- Krav til udvidelserne:
 1. Klienter skal kunne navngive sig selv
 - Dvs. første besked skal indeholde i hvert fald ønskede navn
 2. Når serveren modtager en chat-besked, skal den sende samme besked ud til alle brugere.
 - I følgende format: [NAME]: [MESSAGE]
 3. Hvis klienten sender 'list' til serveren, skal serveren svare med en liste af brugere
 4. Serveren skal annoncere når en ny klient joiner og forlader serveren, til alle brugere

TCP Client/server Øvelse - Hints

- Server modtager 3 typer af beskeder:
 - Join
 - En bruger angiver sit navn
 - Server sender besked til alle
 - Chat
 - En bruger sender en besked
 - Server sender besked til alle
 - List
 - En bruger beder om bruger-liste
 - Server sender besked til anmoder
- Client modtager kun én type besked fra server
 - Skal printes til i konsollen til brugeren
- Hvordan i vælger at håndtere de forskellige beskeder er op til jer selv.
 - Vil dog anbefale en form for serialisering 😊